
AN0025- EC NB-IoT SDK 新手开发指南_V1.1

EigenCOMM Wireless Microcontroller

文档说明

本文档描述移芯通信 NB-IoT 芯片配套的软件 SDK。通过本文档，用户可以了解 SDK 的结构，支持的功能，以及如何上手使用 SDK。

目录

1. 总体介绍	3
1.1 SDK 架构简介	3
1.2 SDK 支持的主要功能	4
1.2.1 NB-IoT modem	4
1.2.2 网络组件	4
1.2.3 外设驱动	6
1.2.4 其他	7
1.3 SDK 目录结构	7
1.3.1 文件夹介绍	8
1.3.2 Middleware 文件夹介绍	8
1.3.3 Project 文件夹介绍	10
2. 使用 ARM 工具链开发	12
2.1 SDK 编译	12
2.1.1 安装 ARM 工具链	12
2.1.2 编译步骤	12
2.2 创建自己的工程	16
2.2.1 利用 SDK 自带示例工程	16
2.2.2 利用 SDK 模板工程	18
2.3 Flash 下载	18
2.3.1 使用 KEIL IDE 下载	18
2.3.2 使用 FLASH 工具下载	21
2.4 使用 EAPT 调试	25
2.4.1 EAPT 安装运行	25
2.4.2 联机在线使用	26
3. 使用 GCC 工具链开发	31
4. 附录	31
4.1 参考文档	31
4.2 术语和解释	32
4.3 安全和警告	33
5. 版本	34
6. 关于我们	错误!未定义书签。

1. 总体介绍

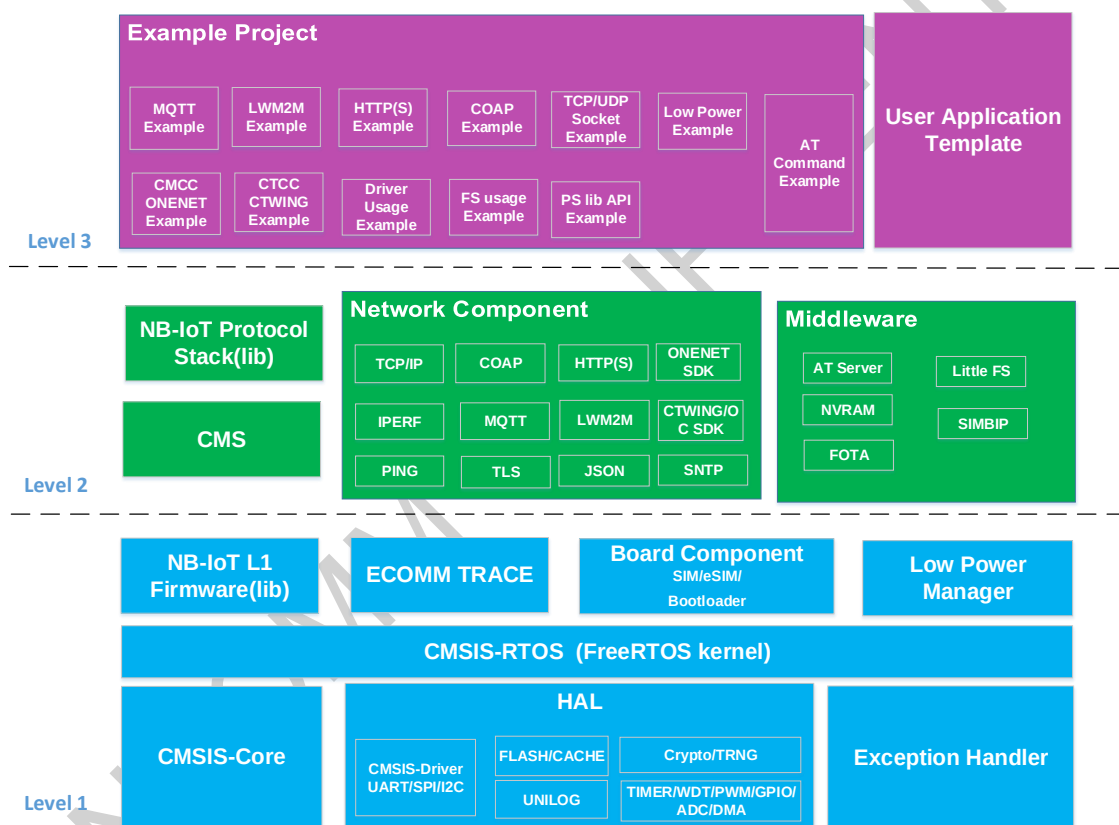
移芯通信 NB-IoT 软件 SDK 提供配套软件和工具，方便开发者开发运行于 NB-IoT 芯片上的应用程序。SDK 包含如下功能：外设驱动，NB-IoT 网络接入能力，TCP/IP/MQTT/COAP/LWM2M/HTTP(S) 等网络协议，Little FS/AT server 等中间件服务，FreeRtos 以及各类 demo 工程等。

本文给出如何使用 SDK 的快速指引，引导用户熟悉 SDK 各功能模块并开始在 SDK 上进行自己的开发工作。

1.1 SDK 架构简介

EC616 SDK 分为 3 层，每层包含不同的组件，整体架构如下图。

Figure 1-1. EC NB-IoT SDK 架构图



下文对 SDK 每层包含的组件进行大致介绍。

Level 1 介绍：

- HAL：包含芯片所支持硬件的驱动程序，向用户提供使用 API。支持的硬件包括：SPI/I2C/UART，这三种总线驱动 API 遵循 ARM CMSIS 驱动标准；TIMER/WDT/PWM/GPIO/ADC/DMA；FLASH/CACHE；UNILog，用于 debug log 的输出。
- CMSIS CORE：遵循 ARM CMSIS 标准，提供 ARM CM3 内部组件的访问 API，如 NVIC, MPU, SYSTICK 等组件的访问。
- RTOS：基于 FreeRTOS 开源实时操作系统内核，提供遵循 ARM CMSIS 标准的 RTOS 功能访问 API。
- ECOMM TRACE：提供调试信息输出 API。
- Exception Handler：用于系统调试，实现异常流程处理，上下文保存等功能。
- Low Power Manager：实现低功耗相关功能。

- **Board Component:** 提供板级组件，如 SIM 驱动，Bootloader 等。
- **NB-IoT L1 Firmware:** 以库文件形式提供 NB-IoT L1 相关功能。

Level 2 介绍：

- **Middleware:** 提供第三方和移芯通信实现的中间件，方便应用开发。主要包括：用于 AT 解析处理的 AT Server 组件，Little FS 开源文件系统，FOTA 以及 SIMBIP 等。
- **Network Component:** 提供各类网络组件，以及第三方主流云平台接入设备侧 SDK。包括：TCP/IP(基于 LWIP)，COAP，LWM2M，HTTP(S)，TLS，JSON，PING，IPERF 等常用网络组件；中国移动，中国电信，阿里云等第三方云平台接入 SDK。
- **CMS: Communication Modem Service** 提供协议交互相关组件，方便用户完成 OPENCPU 相关应用开发。
- **NB-IoT Protocol Stack:** 以库文件形式提供 NB-IoT 网络接入能力，支持 R13 和 R14。

Level 3 介绍：

- **Example Project:** 提供示例工程，方便开发者熟悉 SDK 各组件使用方法，并可以基于示例工程开发自己的应用。包括：驱动使用示例，文件系统使用示例，移动、电信云平台接入示例，AT 使用示例以及各种网络协议使用示例等。
- **User Application Template:** 提供用户工程建立模板，方便用户快速开发自定义应用工程。

1.2 SDK 支持的主要功能

1.2.1 NB-IoT modem

Table 1-1. Modem features

Modem Features
Fully support 3GPP R14 NB-IoT
Category NB2, 2-HARQ
Multi-tone NPUSCH
Anchor and non-anchor carrier
In-band same/different PCI, guard-band, standalone
Multi-carrier paging, NPRACH
Positioning: OTDOA & ECID
ROHC, RAI, multiple-DRB, RRC connection re-establish
SC-PTM (need SW upgrade)

1.2.2 网络组件

EC NB-IoT SDK 支持两类网络组件，一类是独立的标准网络协议，一类是移植的各云平台提供商给出的设备侧 SDK。第二类组件可以省去用户自己移植相关 SDK 的工作，从而可以快速开发相关应用。

下表列出 SDK 支持的标准网络协议及其功能

Table 1-2. 网络功能组件

组件	功能
IP Stack	- IPv4/v6 (LWIP) - TCP, UDP - DNS - SOCKET - SNTP/PING/IPERF
LWM2M	- Eclipse wakaama - LWM2M Server and Client
MQTT	- eclipse paho MQTT, - MQTT 3.1.1, - MQTT client
HTTP(S)	- HTTP1.1 - Client (POST/GET/PUT/DELETE)
COAP	- Libcoap4.2.0 - COAP client - COAPs client - RFC 7252
TLS	- Mbedtls/tinydtls - client - SSL3.0, TLS1.2 - AES, 3DES, DES, ARC4 - MD5, SHA-1, SHA-256 - RSA/PKCS#1 V1.5
JSON	cJSON

下表列出 SDK 支持的云平台。云平台接入功能基于平台提供商提供的设备侧 SDK 移植实现。

Table 1-3. 网络功能组件

云平台	功能
中国移动 ONENET	中国移动物联网公司 ONENET 平台的接入
中国移动 DM	中国移动终端公司设备管理平台 (DMP) 的接入
中国电信 CTWING	中国电信物联网平台的接入

1.2.3 外设驱动

SDK 提供 EC616（617）支持的外设驱动程序，方便用户使用相关外设。

下表列出支持的外设。

Table 1-4. 驱动列表

外设	功能
SPI	- Up to 2 Serial Peripheral Interface - Master/slave mode
UART	- Up to 3 full feature UART - Hardware flow control - Baud rate up to 3M
LPUART	New added feature for chip EC617
I2C	Up to 2 I2C interface
FLASH	4MB SiP - XIP / SW programming - Erase&programme Suspend/Resume
CACHE	16K instruction cache
TIMER	- Up to 6 Timers, - One shot mode - Periodic mode 6 external IO pulse counters
WDT	- Generate interrupt when expired - Trigger system reset when expired
PWM	Up to 6 PWM
GPIO	- Pull up/down - Output/input 6 AON GPIOs in deep sleep mode
AUXADC	4 channel 12 bits AUXADC
UNILog	Hardware /software logging
CRYPTO	Flash encryption

	• AES • SHA1/SHA256
TRNG	• 192 bits truly random number generator
DMA	• 8 channel DMA

1.2.4 其他

下表列出 SDK 支持的其他关键组件

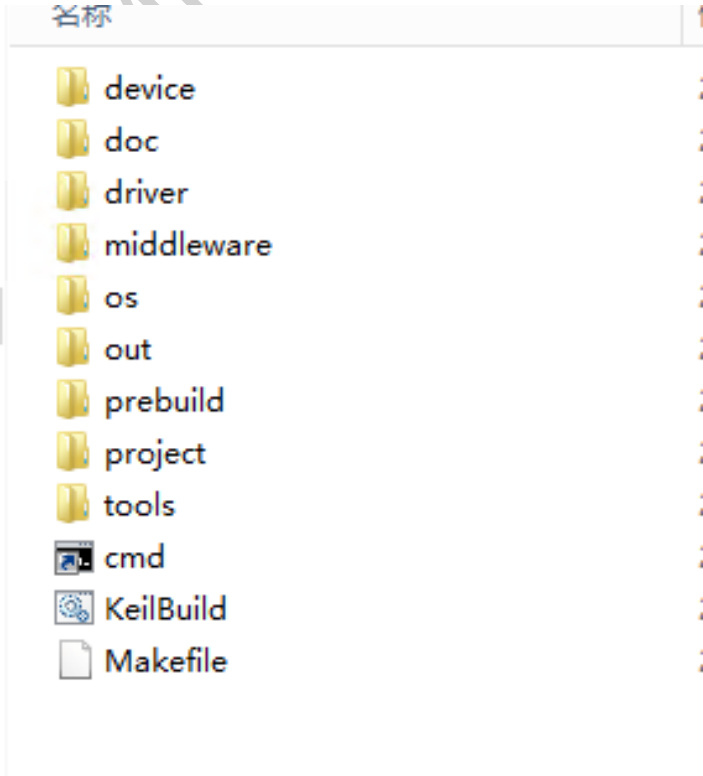
Table 1-5. 其他功能组件

组件	功能
AT Server	• 解析执行 AT 命令，支持标准 AT 命令集，支持自定义 AT 命令添加
LITTLE FS	• Mbed LITTLE FS. high-integrity embedded file system
FOTA	• 基于 OC/ONENET/艾拉比平台的升级方案
SIM BIP	• USIM application toolkit 功能，用于 USIM 卡和远程服务器之间数据传输

1.3 SDK 目录结构

SDK 以压缩包形式发布，解压后顶层文件夹如下图。

Figure 1-2. SDK 顶层文件夹



SDK 中部分功能模块以源代码形式提供，部分功能模块的以库和头文件的形式提供。下面分别介绍每个顶层目录的内容。

1.3.1 文件夹介绍

- **Device:** 放置芯片级别的配置，编译，启动相关的文件。如 ARM linker 需要的 scatter file 文件，芯片启动汇编文件等。
- **Doc 文件夹:** 放置 SDK 相关的介绍文档。如驱动 API 文档，编译指导文档等。其中 readme.txt 描述各文档主要内容，期望作用以及建议阅读顺序。
- **Driver:** EC616 所支持外设的驱动程序。
- **Os:** ARM CMSIS OS 适配层源码，FreeRTOS 源码等。
- **Out:** 编译后生成，原始 SDK release 不存在。存放编译生成的文件，如 *.map, *.bin, *.axf 文件等。
- **Prebuild:** 一些以库和头文件形式发布的模块的相关文件。如 NB-IoT protocol stack 和 L1 模块。
- **Tools:** 常用工具。如使用 KEIL IDE 调试时使用的 flash 下载 pack 包，常用串口调试工具，gcc 编译支持相关工具，log 与解析相关工具等。

下面重点介绍剩余的 2 个顶层文件夹 middleware 和 project。

1.3.2 Middleware 文件夹介绍

Middleware 文件夹包含移芯通信自己实现的 middleware 和移植的开源的第三方 middleware。内容如图 1-3 所示。

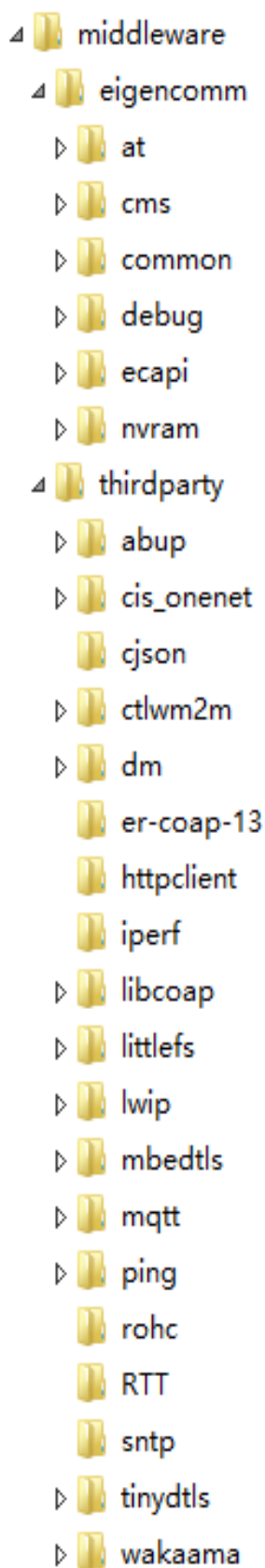
移芯通信自己实现的部分

- **at:** 用于解析和执行 AT 命令。支持用户自定义添加 AT 命令和相关处理函数。
- **cms:** 用于提供协议交互相关组件。
- **common:** 通用文件，例如一些配置文件的维护。
- **Debug:** debug trace 输出 API。
- **Ecap:** 提供给 opencpu 开发的 API
- **nvr:** 用于 NV 文件的维护。

第三方开源移植部分

- **abup:** 艾拉比 FOTA 差分升级方案
- **cis_onenet/ctlwm2m:** 第三方物联网云平台提供商提供的设备侧 SDK，帮助设备快速连接相关云平台。
- **dm:** 运营商认证的自注册功能。
- **Cjson/httpclient/iperf/libcoap/lwip/mqtt/ping/sntp/wakaama:** 第三方开源网络组件。
- **Mbedtls/tinytls:** 数据传输加解密开源组件。
- **Littlefs:** 开源嵌入式文件系统。
- **RTT:** JLINK 辅助 log 输出组件。
- **Rohc:** *add later

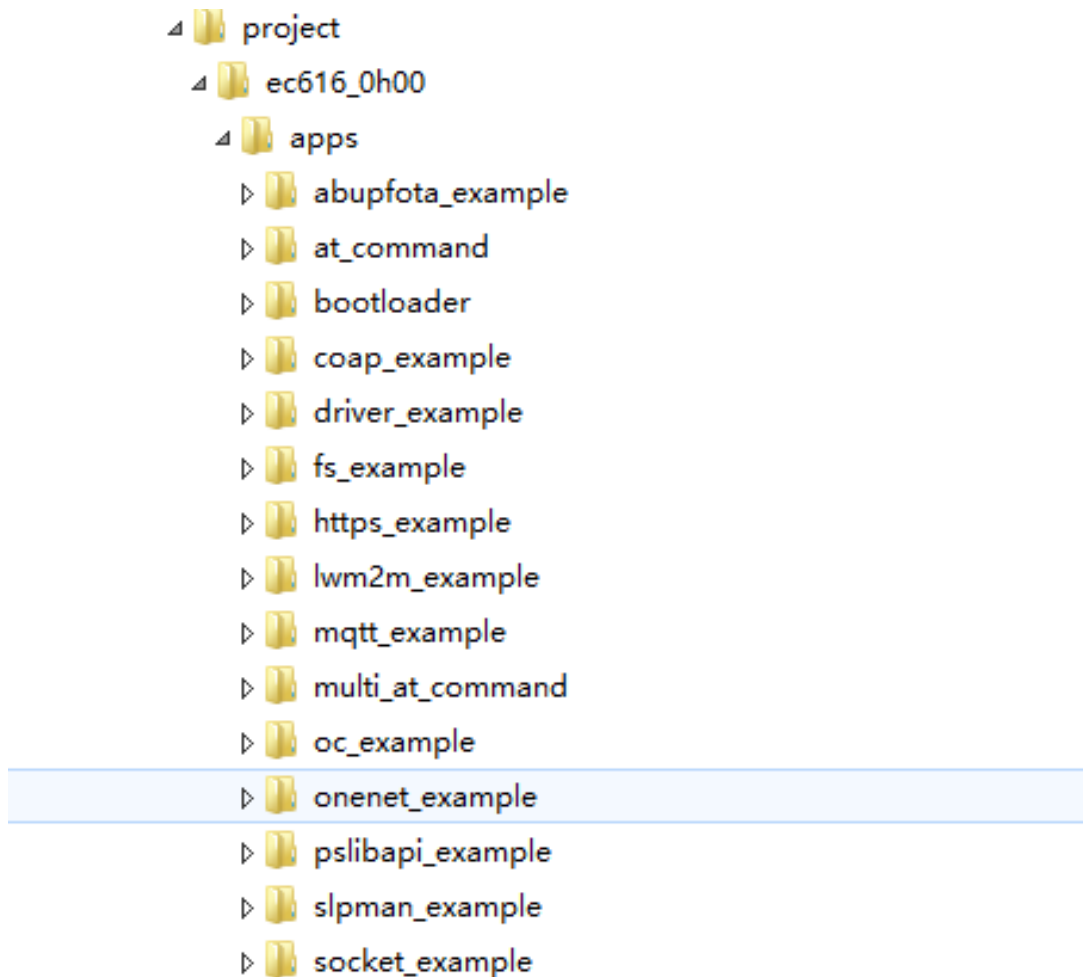
Figure 1-3. EC616 SDK Middleware 文件夹结构



1.3.3 Project 文件夹介绍

Project: 包含 SDK 各种组件的使用示例工程。文件夹结构如图 1-4 所示, 以 EC616 为例。

Figure 1-4. EC616 SDK Project 文件夹结构



- Abupfota_example: 艾拉比 FOTA 升级示例工程。
- At_command: 模组开发默认示例工程。支持通过 AT 命令接口控制模组完成相应通信功能。
- Bootloader: 开放给有修改 bootloader 组件的需求的用户, 默认不开放。
- Coap_example: COAP 接入 server 示例。
- Driver_example: 各驱动模块的使用示例。
- Fs_exmaple: 使用 little FS 存取文件示例。
- https_example: https client 接入 server 示例。
- lwm2m_example: LWM2M 使用示例。
- mqtt_example: MQTT 接入 server 示例。
- multi_at_command: 为少数多串口发送 AT 需求的用户提供示例。
- oc_example/ct_exmaple: 电信 OC 平台接入 example。
- onenet_example: CMCC onenet 平台接入示例。
- pslibapi_example: 协议相关 API 使用示例。

- slpman_example: 低功耗睡眠 API 使用示例。
- socket_example: TCP.UDP 接入 server 示例。

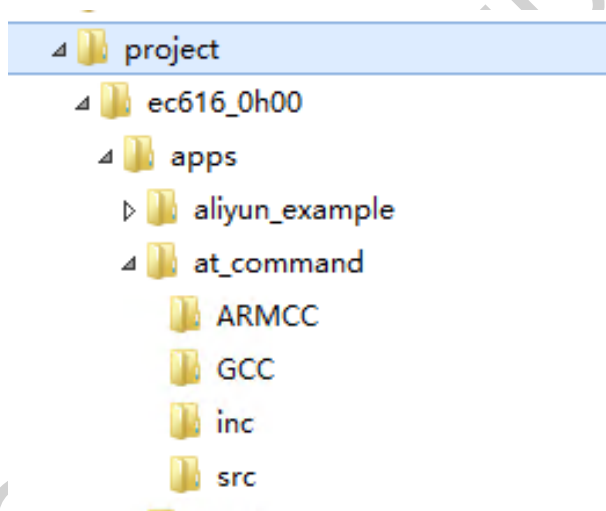
上述示例的具体功能详见各应用手册。

上述示例工程根据使用目的可以分为两类：一类是帮助用户熟悉对应的功能模块，如 driver example；一类本身就是一种应用场景，用户可以选择接近自己需求的示例工程开发，如开发基于 AT 命令交互的模组应用可以选择 at command example，开发中国移动 onenet 平台接入类引用可以选择 onenet example。

根据具体应用是否需要 AT 命令交互也可以分为两类：一类是把 EC616 当做模组，第三方 MCU 或者 PC 通过串口发送 AT 命令实现对 EC616 的控制，如 at command example；一类是基于 EC616 独立做产品的 OPEN CPU 方案，用户可以直接在 EC616 上开发原 MCU 上的相关应用，如 lwm2m example， onenet example 等。

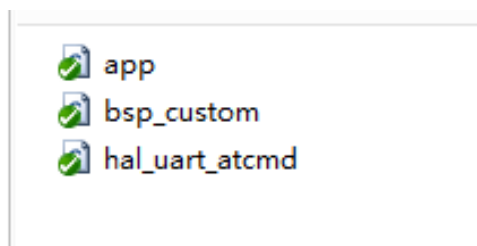
下文以 at_command example 为例进一步展开子文件夹进行介绍，其他示例工程结构相似。

Figure 1-5. At_command 子文件夹结构



- ARMCC 包含 arm tool chain 编译所需的 makefile。
- GCC 包含 gcc tool chain 编译所需的 makefile。*暂时不支持 gcc 编译。
- Inc: 所需的头文件
- Src: 源文件，如图 1-6 所示。App.c 是应用入口文件；bsp_custom.c 是用户对 BSP，如驱动等定制初始化文件；hal_uart_atcmd.c 用于初始化 AT 使用的串口收发通道。

Figure 1-6. At_command/src 细节



2. 使用 ARM 工具链开发

本章节介绍如何基于 ARM 工具链开发和编译 SDK，以及如何运行和 debug 编译出的目标文件。本章主要涉及以下内容：

- 如何编译 SDK 工程
- 如何创建自己的工程
- 如何下载和调试

2.1 SDK 编译

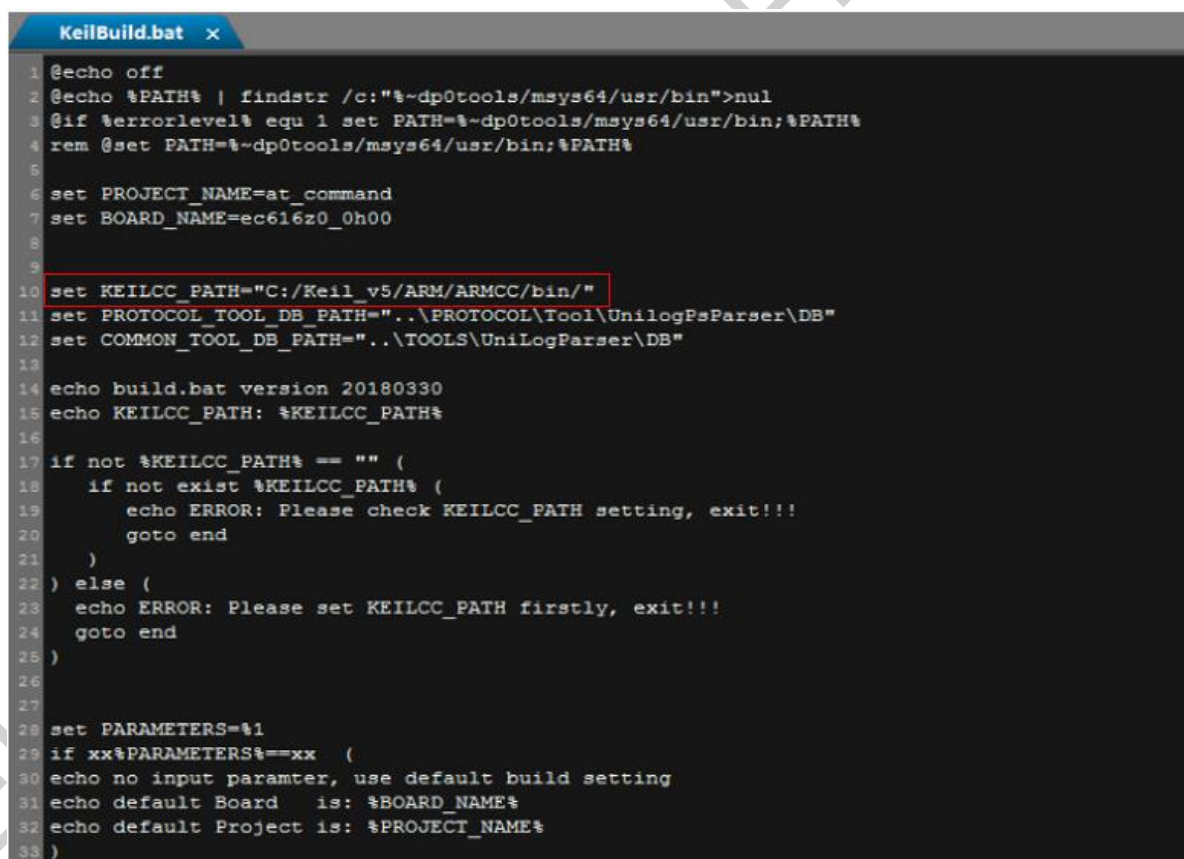
SDK 支持 windows 下 ARM 工具链，本章基于 ARM 工具链进行介绍。

解压 SDK 压缩包，在 SDK 的根目录下会看到支持 SDK 编译的几个 windows bat 脚本，例如 KeilBuid.bat 是支持 ARMCC 编译器的一键编译脚本。

2.1.1 安装 ARM 工具链

用户需要自行安装 keil_v5 IDE，安装成功后 ARM 交叉编译工具链也会被自动安装。然后需要更新 ARM 交叉编译工具链的路径到 SDK 的一键编译脚本 KeilBuid.bat 中。以安装在 C 盘根目录下为例，如下图：

Figure 2-1. ARM 工具链目录

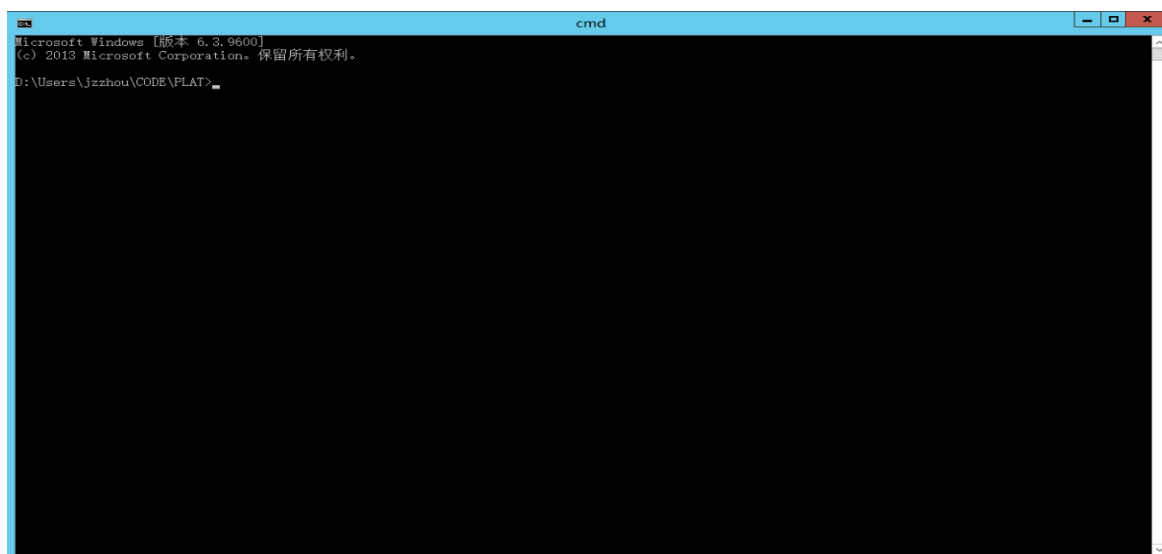


```
1 @echo off
2 @echo %PATH% | findstr /c:"%~dp0tools/msys64/usr/bin">nul
3 @if %errorlevel% equ 1 set PATH=%~dp0tools/msys64/usr/bin;%PATH%
4 rem @set PATH=%~dp0tools/msys64/usr/bin;%PATH%
5
6 set PROJECT_NAME=at_command
7 set BOARD_NAME=ec616z0_0h00
8
9
10 set KEILCC_PATH="C:/Keil_v5/ARM/ARMCC/bin/"
11 set PROTOCOL_TOOL_DB_PATH="..\PROTOCOL\Tool\UnilogPsParser\DB"
12 set COMMON_TOOL_DB_PATH="..\TOOLS\UniLogParser\DB"
13
14 echo build.bat version 20180330
15 echo KEILCC_PATH: %KEILCC_PATH%
16
17 if not %KEILCC_PATH% == "" (
18     if not exist %KEILCC_PATH% (
19         echo ERROR: Please check KEILCC_PATH setting, exit!!!
20         goto end
21     )
22 ) else (
23     echo ERROR: Please set KEILCC_PATH firstly, exit!!!
24     goto end
25 )
26
27
28 set PARAMETERS=%1
29 if %PARAMETERS%==xx (
30     echo no input paramter, use default build setting
31     echo default Board is: %BOARD_NAME%
32     echo default Project is: %PROJECT_NAME%
33 )
```

2.1.2 编译步骤

步骤 1：双击 SDK 根目录下的 cmd.exe，进入 windows 命令程序。

Figure 2-2. Cmd 命令行



步骤 2: 运行 KeilBuild.bat 批处理文件, 该批处理文件支持通过传入参数来控制不同的编译行为, 该参数是可选项, 如用户不指定该传入参数, KeilBuild.bat 会按照默认的行为去编译默认板级下的默认工程, 如指定就会按照参数指定的行为去编译特定的工程。传入参数的类型及格式为: KeilBuild.bat **【BoardName-ProjectName-Option】**。

注: BoardName/ ProjectName/ Option 之间使用的是“-”中划线进行连接的。

在 SDK 中, 对于 EC616 默认 BoardName 为 ec616_0h00, 对于 EC617 默认 BoardName 为 ec617_0h00。一般无需用户无需特别指定 BoardName

具体参数介绍如下表 (以 EC616 为例):

Table 2-1. 编译参数列表

参数名称	参数选项	参数说明
BoardName	ec616_0h00	EC616 DKB 板。 默认选项
	At_command	模组开发默认示例工程。支持通过 AT 命令接口控制模组完成相应通信功能。 默认选项
	Coap_example	COAP 接入 server 示例。
	Driver_example	各驱动模块的使用示例。
	Others	其他工程
Option	clean	Clean 当前编译工程
	clall	Clean 所有工程
	*unilog	编译前预解释生成新的 debug_log.h
	doc	生成文档
	verbose	编译过程输出详细的编译命令 log
	help	输出帮助信息

	list	列出所有支持的板子和工程
--	------	--------------

*注：第一次编译或者增加删除或修改 log 后需要传入 unilog 参数进行编译。因为 log 需要预处理并生成对应的 debug_log.h 文件，此文件应随着 log 改变而改变并参与后续编译。

用户也可以通过运行#Keil_build.bat help 来查看所有支持的命令参数以及用法。

Figure 2-3. Help 选项输出

```

Microsoft Windows [版本 6.3.9600]
(c) 2013 Microsoft Corporation. 保留所有权利。

D:\Users\jzzhou\CODE\PLAT>KeilBuild.bat help
build.bat version 20180330
KEILCC_PATH: "C:/Keil_v5/ARM/ARMCC/bin/"
OPTION: help
=====
-----Show How to Build Project-----
=====
                                Keilbuild.bat Board-Project-Option
Board,Project,Option could be omitted, if so default Board and project will be used
=====
Supported Options:
doc---run doxygen to generate documents
unilog---pre-parse the log before compile
clean---clean the build output files
clall---clean all output files for every board and project
jenkins_release---trigger jenkins_release
list---list current supported boards and supported project for each board
help---show how to use the build script
=====
Example start -- Example start -- Example start -- Example start
-----
Keilbuild.bat  ec616z0_0h00-at_command"
build at_command project for board ec616z0_0h00"
=====
Keilbuild.bat  ec616_0h00-driver_example"
build driver_example project for board ec616_0h00"
=====
Keilbuild.bat  ec616_0h00-bootloader"
build bootloader project for board ec616_0h00"
=====
Keilbuild.bat  ec616_0h00-driver_example-clean"
clean driver_example project for board ec616_0h00"
=====
Keilbuild.bat clean"
clean default project for default board
=====
Keilbuild.bat clall"
clean all output files for every board and project
=====
Keilbuild.bat unilog"
perform log preparse of default project for default board
=====

```

编译结束后，cmd 窗口中会输出此次编译的结果，如图 2-4 所示为编译成功的输出

Figure 2-4. 编译成功结果输出

```
LINK ../../../../../../PLAT/out/ec616z0_0h00/onenet_example/onenet-demo-flash.axf
"../../../../../../../../PLAT/device/eigencomm/board/ec616z0_0h00/boot/keil/ec616_0h00_flash.sct", line 61 (column 14): Warning:
pattern bsp_spi.o(RW).
"../../../../../../../../PLAT/device/eigencomm/board/ec616z0_0h00/boot/keil/ec616_0h00_flash.sct", line 61 (column 18): Warning:
pattern bsp_spi.o(ZI).
"../../../../../../../../PLAT/device/eigencomm/board/ec616z0_0h00/boot/keil/ec616_0h00_flash.sct", line 66 (column 17): Warning:
pattern gpio_ec616.o(RW).
"../../../../../../../../PLAT/device/eigencomm/board/ec616z0_0h00/boot/keil/ec616_0h00_flash.sct", line 66 (column 21): Warning:
pattern gpio_ec616.o(ZI).
"../../../../../../../../PLAT/device/eigencomm/board/ec616z0_0h00/boot/keil/ec616_0h00_flash.sct", line 78 (column 16): Warning:
pattern pmu_ec616.o(RW).
"../../../../../../../../PLAT/device/eigencomm/board/ec616z0_0h00/boot/keil/ec616_0h00_flash.sct", line 78 (column 20): Warning:
pattern pmu_ec616.o(ZI).
"../../../../../../../../PLAT/device/eigencomm/board/ec616z0_0h00/boot/keil/ec616_0h00_flash.sct", line 79 (column 18): Warning:
pattern timer_ec616.o(RW).
"../../../../../../../../PLAT/device/eigencomm/board/ec616z0_0h00/boot/keil/ec616_0h00_flash.sct", line 79 (column 22): Warning:
pattern timer_ec616.o(ZI).
"../../../../../../../../PLAT/device/eigencomm/board/ec616z0_0h00/boot/keil/ec616_0h00_flash.sct", line 85 (column 5): Warning:
pattern *(.mcuData).
"../../../../../../../../PLAT/device/eigencomm/board/ec616z0_0h00/boot/keil/ec616_0h00_flash.sct", line 100 (column 5): Warning:
pattern *(.mcuCode).
Program Size: Code=1308856 RO-data=71400 RW-data=16908 ZI-data=146176
Finished: 0 information, 10 warning and 0 error messages.
make[1]: Leaving directory '/cygdrive/d/Users/jzzhou/CODE/PLAT/project/ec616z0_0h00/apps/onenet_example/ARMCC'

End time: 2019/03/25 周一 14:36:19.24
Built took 0 hours 0 minutes 57 seconds
#####
##                                     ##
##          #####          ##          ##          ##          ##          ##
##          ##  ##  ##  ##  ##  ##  ##  ##          ##
##          ##  ##  ##  ##  ##  ##  ##  ##          ##
##          #####  ##  ##  #####          ##
##          ##          #####          ##          ##
##          ##          ##  ##  ##  ##  ##          ##
##          ##          ##  ##  ##  #####          ##
##          ##          ##          #####          ##
##          #####
#####
build successfully
```

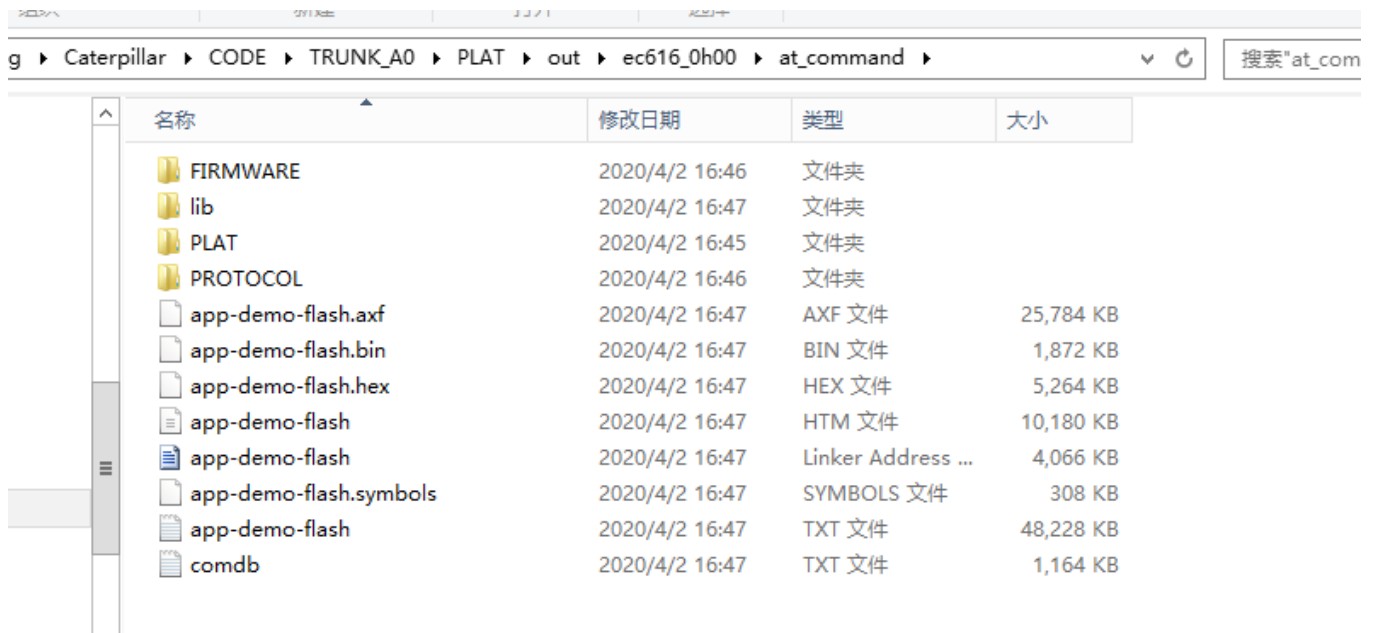
如果编译失败，用户应该根据编译 log 查找问题点，解决并重新编译。

对于编译输出的中间文件，库文件，工程执行 axf/hex 文件，map 文件以及对应的反汇编文件，都会按照不同的板级和工程配置输出到 SDK/out 下不同的子目录下。如图 2-5 和图 2-6 所示：

Figure 2-5. 编译结果输出目录

ea ▶ bchang ▶ Caterpillar ▶ CODE ▶ TRUNK_A0 ▶ PLAT ▶ out ▶ ec616_0h00 ▶			
名称	修改日期	类型	大小
at_command	2020/4/2 16:47	文件夹	

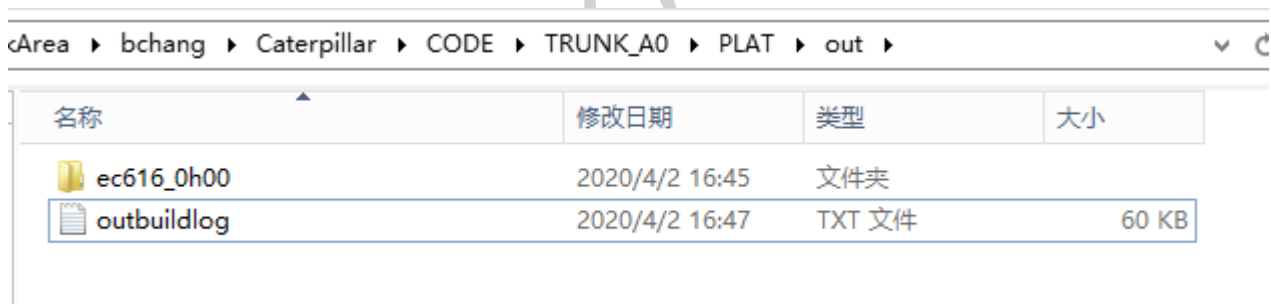
Figure 2-6. 编译输出文件



名称	修改日期	类型	大小
FIRMWARE	2020/4/2 16:46	文件夹	
lib	2020/4/2 16:47	文件夹	
PLAT	2020/4/2 16:45	文件夹	
PROTOCOL	2020/4/2 16:46	文件夹	
app-demo-flash.axf	2020/4/2 16:47	AXF 文件	25,784 KB
app-demo-flash.bin	2020/4/2 16:47	BIN 文件	1,872 KB
app-demo-flash.hex	2020/4/2 16:47	HEX 文件	5,264 KB
app-demo-flash	2020/4/2 16:47	HTM 文件	10,180 KB
app-demo-flash	2020/4/2 16:47	Linker Address ...	4,066 KB
app-demo-flash.symbols	2020/4/2 16:47	SYMBOLS 文件	308 KB
app-demo-flash	2020/4/2 16:47	TXT 文件	48,228 KB
comdb	2020/4/2 16:47	TXT 文件	1,164 KB

对于编译过程中的 log 文件，包括错误信息，生成于 SDK/out 目录下，文件名为 outbuildlog.txt:

Figure 2-7. 编译 log 文件



名称	修改日期	类型	大小
ec616_0h00	2020/4/2 16:45	文件夹	
outbuildlog	2020/4/2 16:47	TXT 文件	60 KB

2.2 创建自己的工程

用户可以使用 2 种方法来创建自己的工程。

2.2.1 利用 SDK 自带示例工程

此方法适用于用户期望开发的功能与 SDK 自带示例工程相似的情况。比如用户希望开发基于 AT 命令交互的应用并添加自定义的命令和相关处理，就可以直接使用 at command example 进行二次开发。又或者用户希望开发 onenet 接入类应用，就可以基于 onenet example 进行二次开发。

下面以 at command example 为例进行说明。

步骤 1: 将 at command example 拷贝到 project/ ec61x_0h00/apps 下并重命名，如 my_project

步骤 2: 在根目录下的 keilbuild.bat 增加对 my_project 的编译支持。

Figure 2-8. 添加编译工程

```
echo PROJECT_NAME: %PARAMETERS% | findstr "mqtt_example"
if not errorlevel 1 (
    set PROJECT_NAME=mqtt_example
)

echo PROJECT_NAME: %PARAMETERS% | findstr "coap_example"
if not errorlevel 1 (
    set PROJECT_NAME=coap_example
)

echo PROJECT_NAME: %PARAMETERS% | findstr "dm_example"
if not errorlevel 1 (
    set PROJECT_NAME=dm_example
)

echo PROJECT_NAME: %PARAMETERS% | findstr "my_project"
if not errorlevel 1 (
    set PROJECT_NAME=my_project
)
```

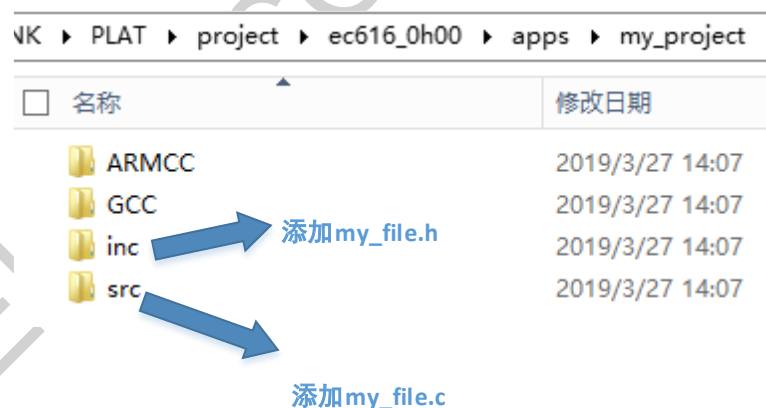
步骤 3: 自定义是否编译相关功能模块, 如果 my_project 不需要某些功能或者需要添加新的功能模块可以通过修改 project/ /ec61x_0h00/ apps/my_project/ARMCC/ makefile 实现是否编译。如支持 AT 组件, 但不使能 AT debug 组件可以做如下设置:

```
BUILD_AT = y
BUILD_AT_DEBUG = n
```

关于可配置组件的详细说明见 AN0020- EC NB-IoT SDK 编译指南.pdf, 本文不做展开。

步骤 4: 添加自定义文件。在对应的 inc 目录添加头文件, 在 src 目录添加源文件。

Figure 2-9. 添加文件



文件添加完后需要修改 makefile 增加对新增源文件的编译支持。

Figure 2-10. 添加编译支持

```
AVAILABLE_TARGETS = ec616_0h00 ec616z0_0h00
TOOLCHAIN          = ARMCC
BINNAME            = app-demo-flash
TOP                := ../../../../..

CFLAGS_INC         += -I ../inc
obj-y              += PLAT/project/$(TARGET)/apps/at_command/src/app.o \
                     PLAT/project/$(TARGET)/apps/at_command/src/bsp_custom.o \
                     PLAT/project/$(TARGET)/apps/at_command/src/user_config.o \
                     PLAT/project/$(TARGET)/apps/at_command/src/my_file.o

include $(TOP)/PLAT/tools/scripts/Makefile.rules

ifneq ($(BUILD_AT),y)
$(error This example needs to modify "BUILD_AT" to "y" in device\eigencomm\board\$(TARGET)\ec616_0h00.mk)
endif
```

2.2.2 利用 SDK 模板工程

用户也可以选择使用 project/ec61x_0h00/templates 中模板工程进行自定义开发，使用步骤与使用示例工程类似，但模板工程没有包含具体功能演示，只是一个开发模板。用户应用如果和某个示例工程接近，建议用户使用方法 1 来开发。

步骤 1: 将 templates 下 app_demo 工程 拷贝到 project/ ec61x_0h00/apps 下并重命名，如 my_project

步骤 2: 在根目录下的 keilbuild.bat 增加对 my_project 的编译支持。

*步骤 3: 自定义是否编译相关功能模块。由于只是模板工程，用户需要阅读对应编译文档细节，了解开配置是否编译的功能模块，确定需求并在 makefile 做相应配置。

步骤 4: 添加自定义文件。在对应的 inc 目录添加头文件，在 src 目录添加源文件。

*关于步骤 3: 如果使用方法 1 的演示工程，大部分功能模块是否编译的配置都已经预先配置好，以便使用相应功能，用户无特殊需求不必自行增减。但如果使用方法 2 的模板工程，则用户需要明确需要使用的模块。

2.3 Flash 下载

支持 2 种下载方式，分别是使用 KEIL IDE 下载和使用 flash 下载工具下载。前者适何希望使用 KEIL IDE 调试的场景，后者适合仅使用 log 调试的场景，一般对应底层相对稳定后的场景。

2.3.1 使用 KEIL IDE 下载

SDK 支持使用 KEIL IDE 下载和调试。本文仅做下载相关的介绍。使用 KEIL IDE 调试详见具体文档。

此下载方式适用于希望利用 KEIL IDE 开发调试的开发者，如需要单步，断点等调试功能。

使用 KEIL IDE 下载需要先安装 keil pack，然后可以通过 KEIL 界面触发下载。

本文以 EC616 为例进行介绍。

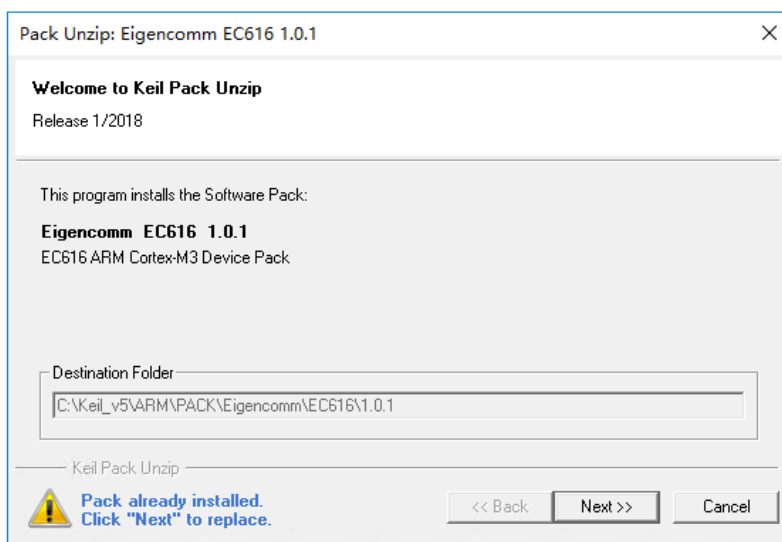
EC616 keil pack 包位于 tools/keil-pack 下，双击即可安装（需预装 keil-v5）。

Figure 2-11. Keil pack 目录

名称	修改日期	类型	大小
EC616.1.0.4	2020/4/1 9:26	7Z 文件	1,154 K
Eigencomm.EC616.1.0.4	2020/4/1 9:26	uVision Software...	33 K

步骤 1: 安装 pack 包, 点击如下图 next 即可以安装。

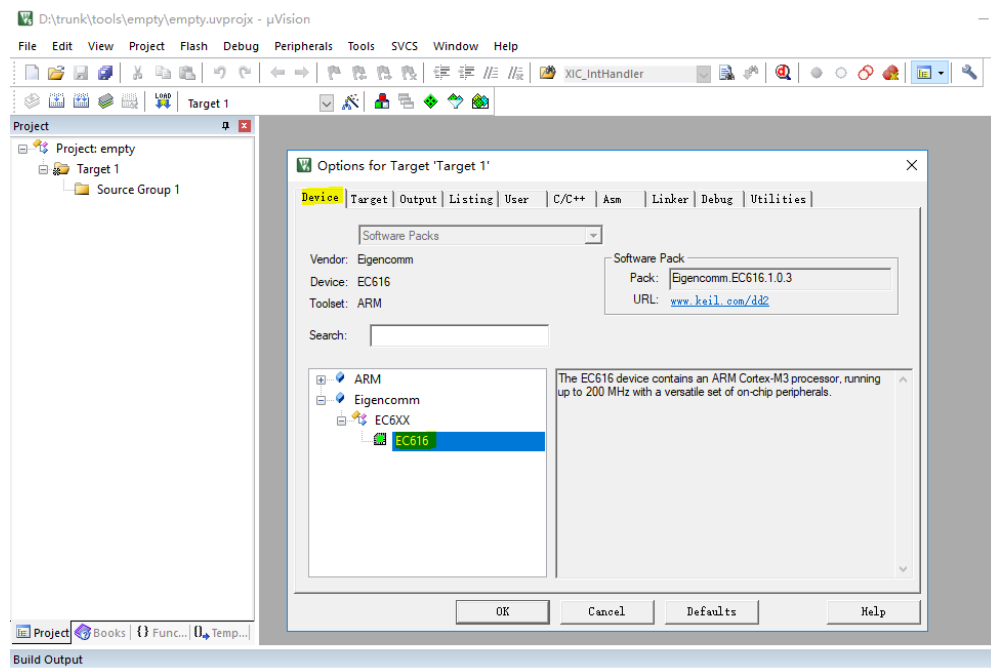
Figure 2-12. Keil pack 安装



步骤 2: 下载相关配置

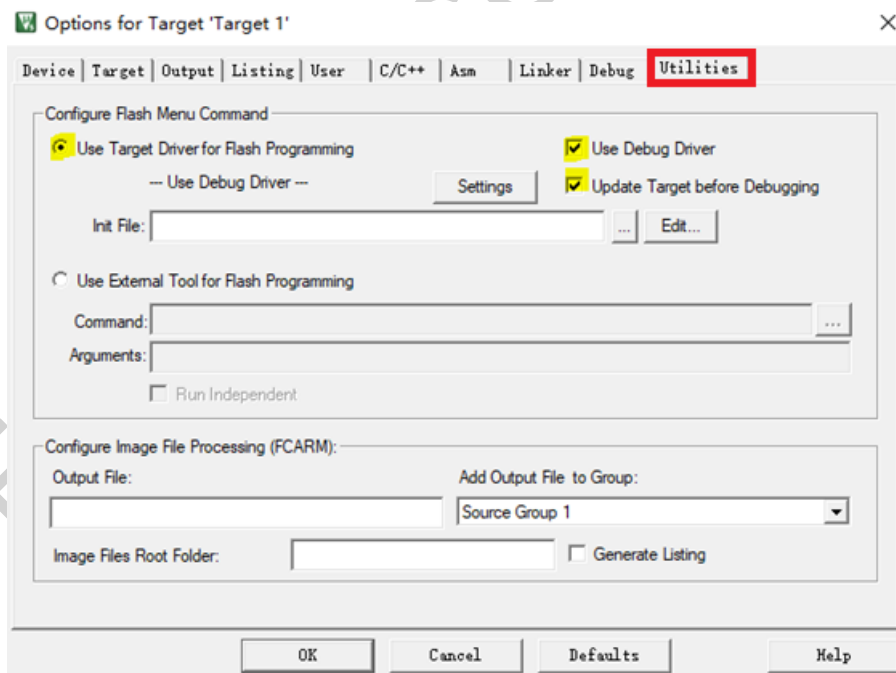
- 1) 在 device 页面选择 ec616, 如下图所示

Figure 2-13. 选择 device



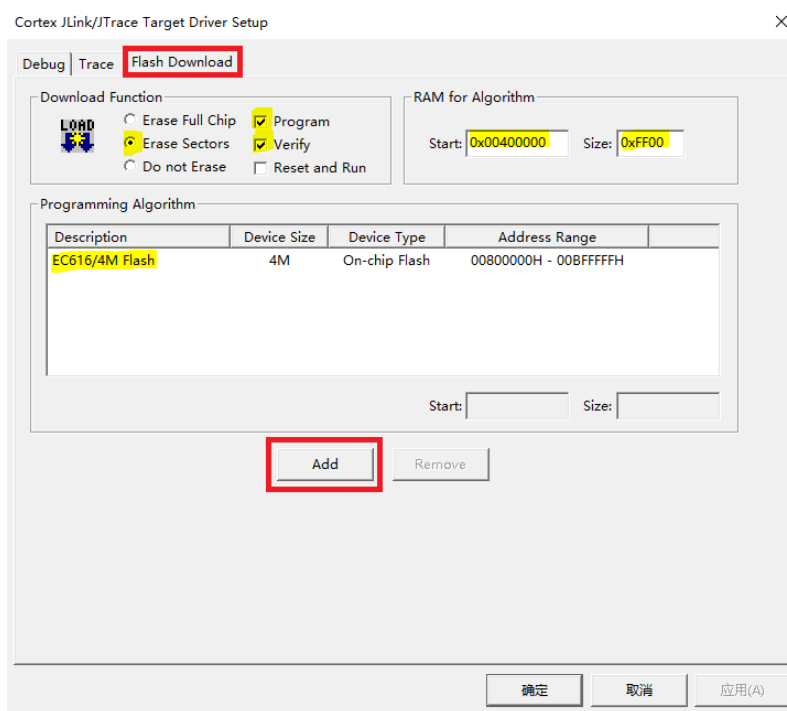
2) 在 utilities 页面做如下图选择，点击 settings 按钮进入 flash 配置框。

Figure 2-14. Utilities 设置



3) 弹出如下图所示 flash 对话框后，选择 flash download 页，点击 add 按钮增加 EC616 对应的 flash 配置，按照高亮所示进行修改。

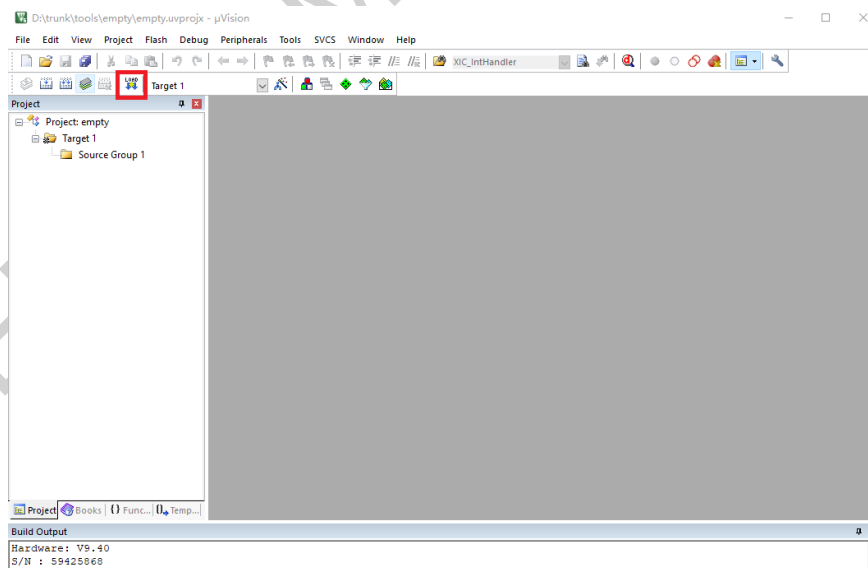
Figure 2-15. flash 设置



步骤 3: 触发下载。

配置完成后点击确定。点击如下图所示快捷按钮进行 axf 下载。

Figure 2-16. 触发下载

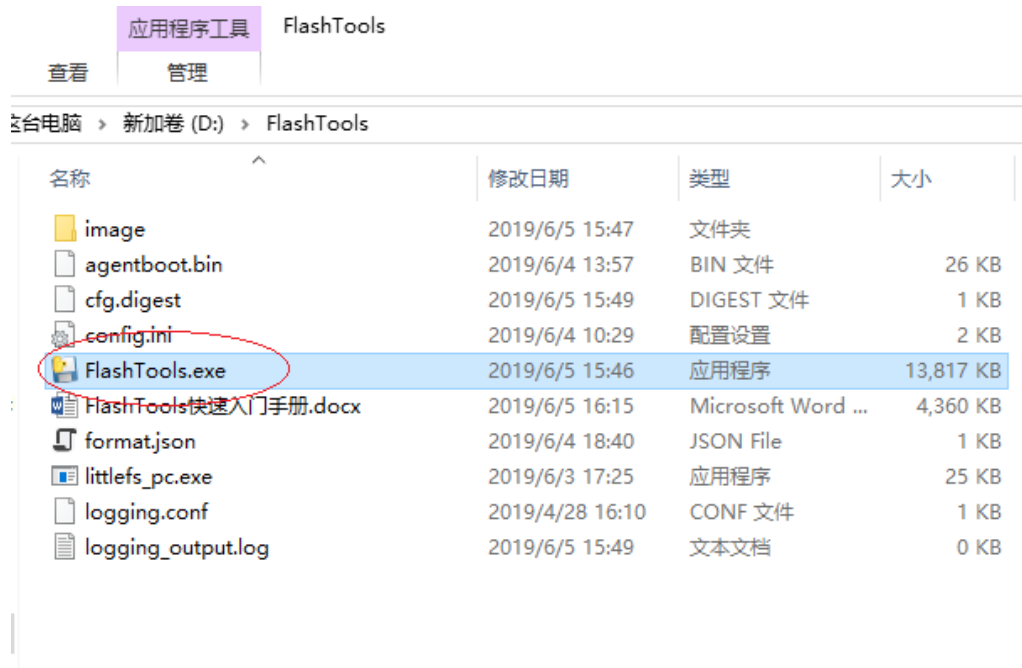


2.3.2 使用 FLASH 工具下载

步骤 1: 运行下载软件

FlashTools 绿色安装，解压后即可使用。在解压后的目录下，运行 FlashTools.exe。用户也可参考 FlashTools 自带文档。

Figure 2-17. Flashtools 可执行文件



步骤 2: 进入下载模式

必须正确设置 UE 使其进入下载模式，Flashtools 软件才能正确运行。下图仅以第一版 DKB 为例。

其他不同板子可参考具体的文档。

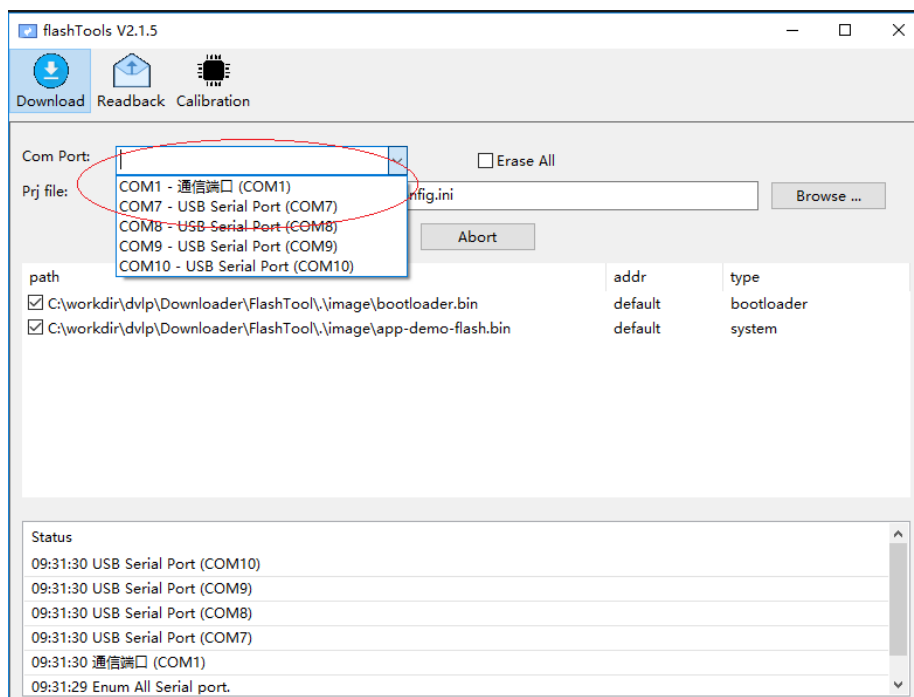
Figure 2-18. 进入下载模式



如上图所示，在上电后，先按下 SOC_BT1，然后按下 SOC_RST#，然后先释放 SOC_RST#，再释放 SOC_BT1，进入 SOC 下载模式。

步骤 3: 选择下载 COM 口

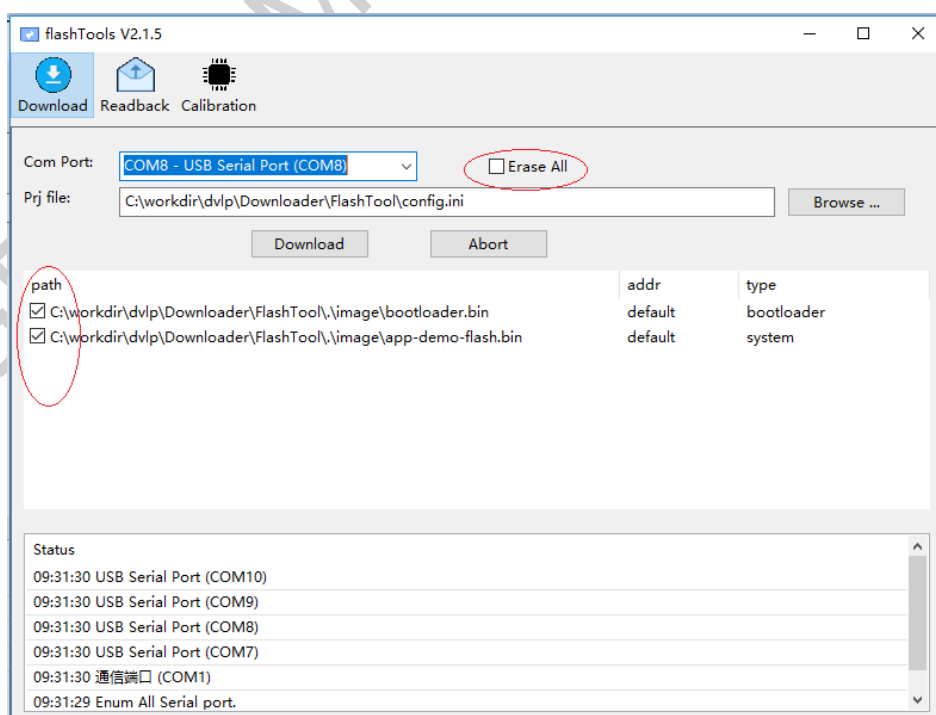
Figure 2-19. COM 口选择



步骤 4: 烧写文件

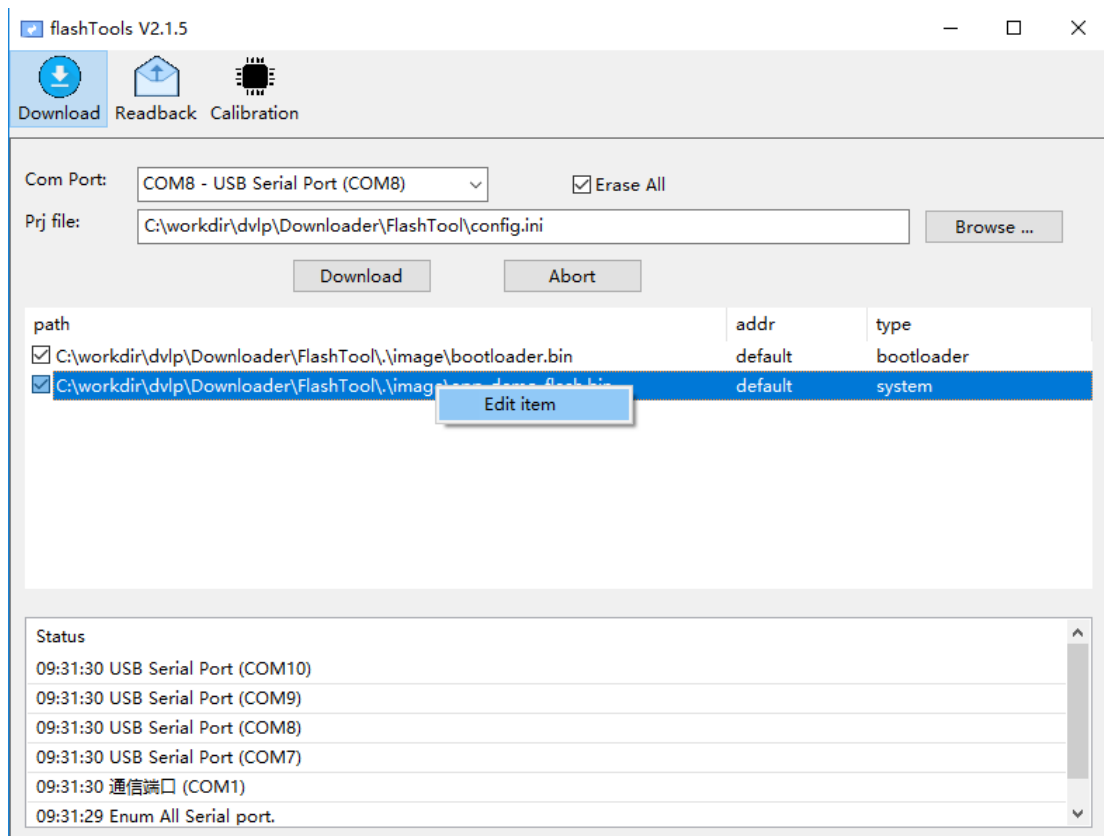
在文件列表中，勾选准备烧写的文件。如果 Erase 整个 flash，请勾选界面中的 Erase All 选项框。

Figure 2-20. 下载设置



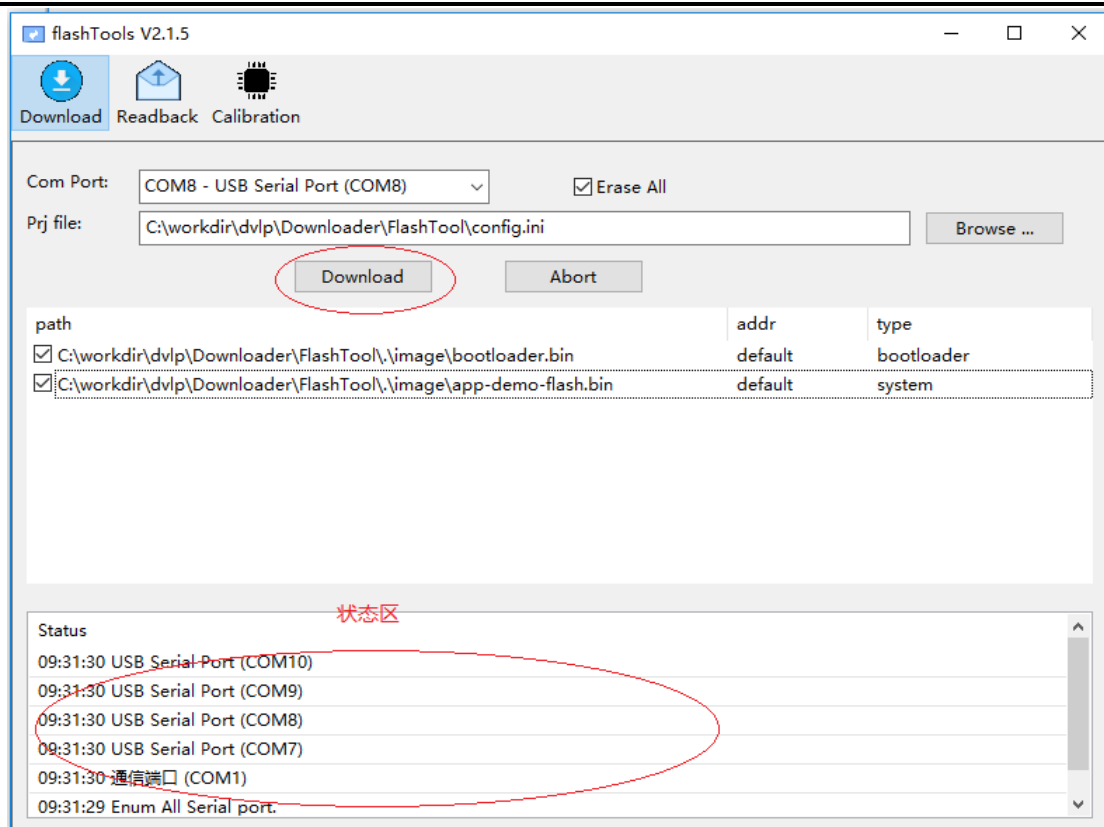
如果想编辑单个下载文件，请选中列表中的某个文件，鼠标右键弹出菜单 Edit Item，点击此菜单可以选择指定目录下的下载文件。

Figure 2-21. 选择待下载文件



点击 download 按钮，开始下载，通过界面上的状态区可以查看下载的状态。

Figure 2-22. 开始下载



2.4 使用 EPAT 调试

EPAT 是用于分析、调试、命令等功能的工具或者工具集合。

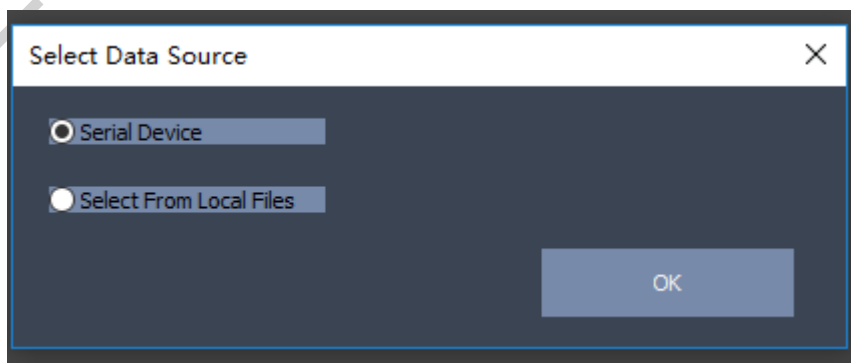
本文仅作入门介绍，详细使用文档参考 EPAT 调试工具使用手册。

2.4.1 EPAT 安装运行

EPAT 为绿色安装，解压后即可使用。

在安装目录\EPAT\bin 目录下，运行 EPAT.exe，首先要启动模式选择启动后将弹出模式选择框，根据需求选择适合你的 mode。

Figure 2-23. EPAT 模式设置



如果联机捕获 UE Log 请选择 Serial Device。

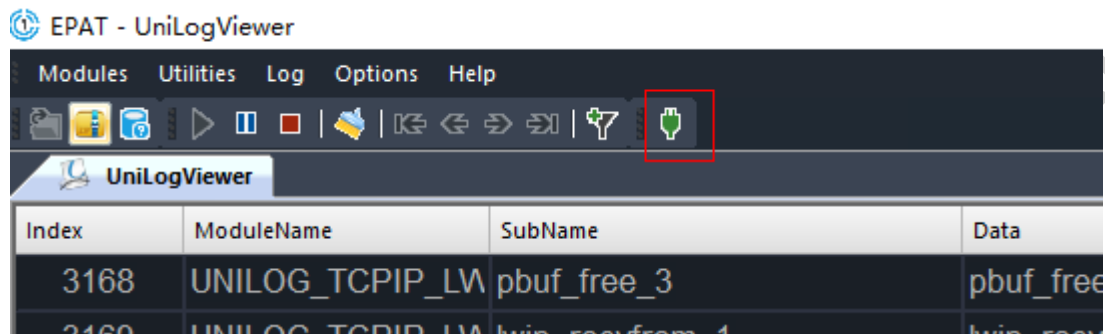
如果查看保存的 zip log，请选择 Select From Local Files。

2.4.2 联机在线使用

步骤 1：配置 log 串口和波特率。由于串口号无法设置成固定值，使用者需明确用于 log 输出的串口号。

在主界面的工具栏点击 Device Communication 按钮。

Figure 2-24. 串口设置-1



将会弹出 Device 设置对话框。

Figure 2-25. 串口设置-2

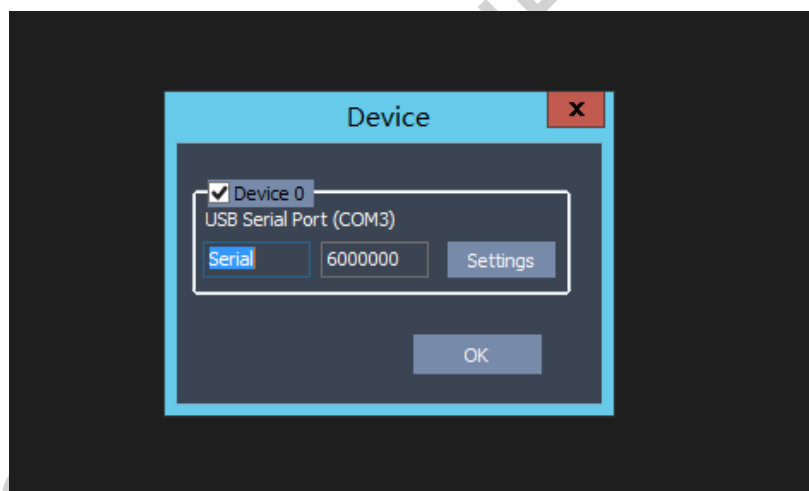
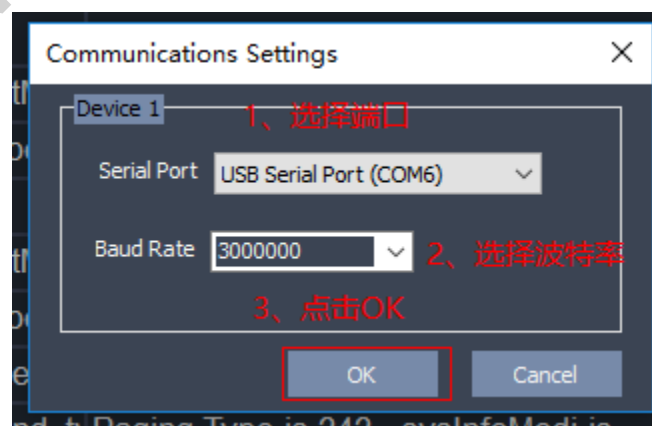


Figure 2-26. 串口设置-3



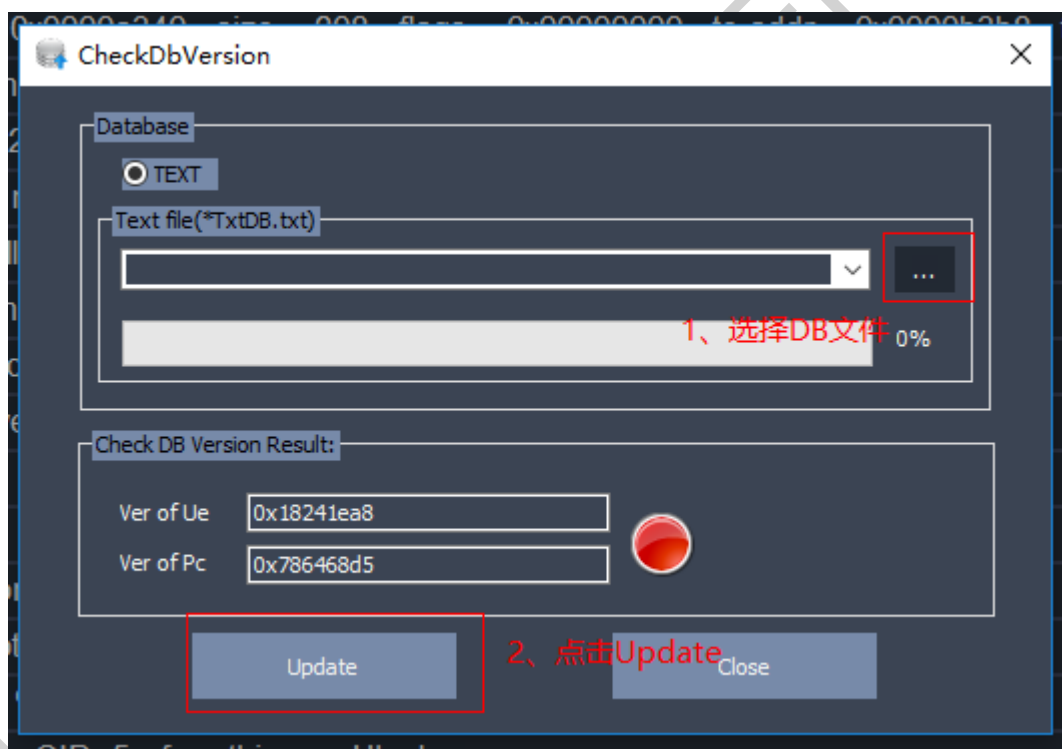
步骤 2：更新数据库

点击右上角 database 按钮



弹出更新数据库界面，手动更新你的数据库文件。

Figure 2-27. 数据库更新



更新后，上图 Check DB Version Result 区域显示结果：

绿色：成功且匹配

红色：不匹配

灰色：读取 UE DB 失败。

步骤 3：log 抓取。配置并连接成功后，EAPT 会显示输出的 log 信息

EPAT - UniLogViewer - Log_20181204_171823.ulf

Modules

Utilities

Log

Options

Help

UniLogViewer

Index	ModuleName	SubName	Data
0			08 76 00 00 00 00 00 00 00 00 00 00 00 00 7F 00 00 FF 00 00 00 0C 08 58 02 6A 80 84 DC 00
1	UNILog_MAC	CeMacUIOneLcScheduling_2	UL LCID: 3 ; assign grant: 122
2	UNILog_RLC	CeRlcAmUIAssembleRetxPdu_2	AM UL , LC: 3 , too much RLC PDU () construct this time , can ' t construct any more
3	UNILog_PS_DUMP	CedrDIUserPlaneDataInd_1	CEDR DL IP PKG via UP: 3D 03 11 99 E7 27 2C 7E F3 B2 EE 52 00 EB 00 E4 45 00 0
4	UNILog_MAC	CeMacRaRandomAccessSucc_	PRACH SUCC
5			09 4A 09 4A 98 08 00 0C 10 00 02 00 00 05 27 01 03 00 01 00 00 12 43 32 00 45 41 9
6	UNILog_MAC	CeMacUIOneLcScheduling_2	UL LCID: 3 ; assign grant: 124
7	UNILog_RLC	CeRlcAmUIAssembleRetxPdu_2	AM UL , LC: 3 , too much RLC PDU () construct this time , can ' t construct any more
8	UNILog_RLC	CeRlcAmUIAssembleStatusPdu_	RLC AM , LC: 3 , not enough grant: 119 , to ASSEM status PDU
9	UNILog_RLC	CeRlcAmCheckSetPBit_1	AM UL ; LC: 3 ; no more UL data pending: new_retx
10	UNILog_PS_DUMP	CedrDIUserPlaneDataInd_1	CEDR DL IP PKG via UP: 3F 3D 03 00 B1 E8 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D
11	UNILog_PS_DUMP	CeMacUISegPduCheck_1	MAC UL PDU: 23 02 1F 07 A4 6F AF F0 30 70 B0 F1 31 71 B1 C0 9F
12	UNILog_PDCP	CCePdcpUITransSuccInd_2	PDCP_SDU Timing delay can not be calculated:curTicks 3081 , oldTicks 3 , lcId
13	UNILog_RLC	CeRlcAssembleLcDisdu_1	RLC AM LC: 3 , don ' t has valid PDCP PDU , maybe only RLC STATUS report , or RLC
14	UNILog_MAC	CeMacDIProcCephyDIDataIndS	One DL RLC PDU , LCID: 3 Len: 2
15	UNILog_CERRC	CerrcGetCellBarStatusFromSib	Cell is barred due to SIB1->cellBarred is TRUE! \n
16	UNILog_PS_DUMP	CeMacUISegPduCheck_1	MAC UL PDU: 23 51 1F 89 67 04 20 3B C9 E6 B6 64 56 34 94 80 3A C0 39 11 41 00 3
17	UNILog_MAC	CeMacDIProcCephyDIDataIndS	One DL RLC PDU , LCID: 3 Len: 81

<

Log Position: 0.0% | Page:1/305

>

Device 0 | Device 1



选择工具栏的 Save 按钮或者 CTRL+S 快捷键，弹出保存 Log 对话框。

Figure 2-29. Log 保存-1

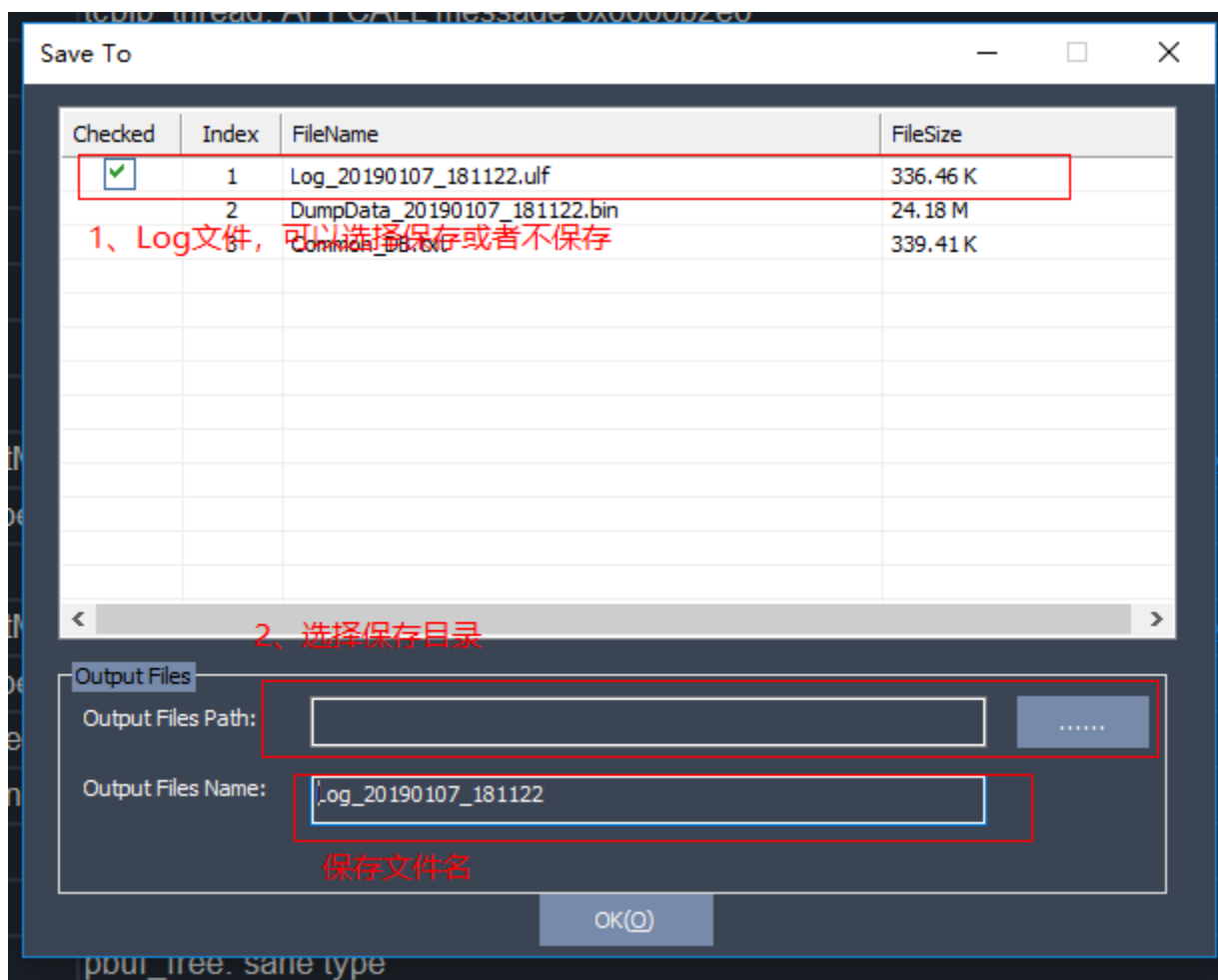
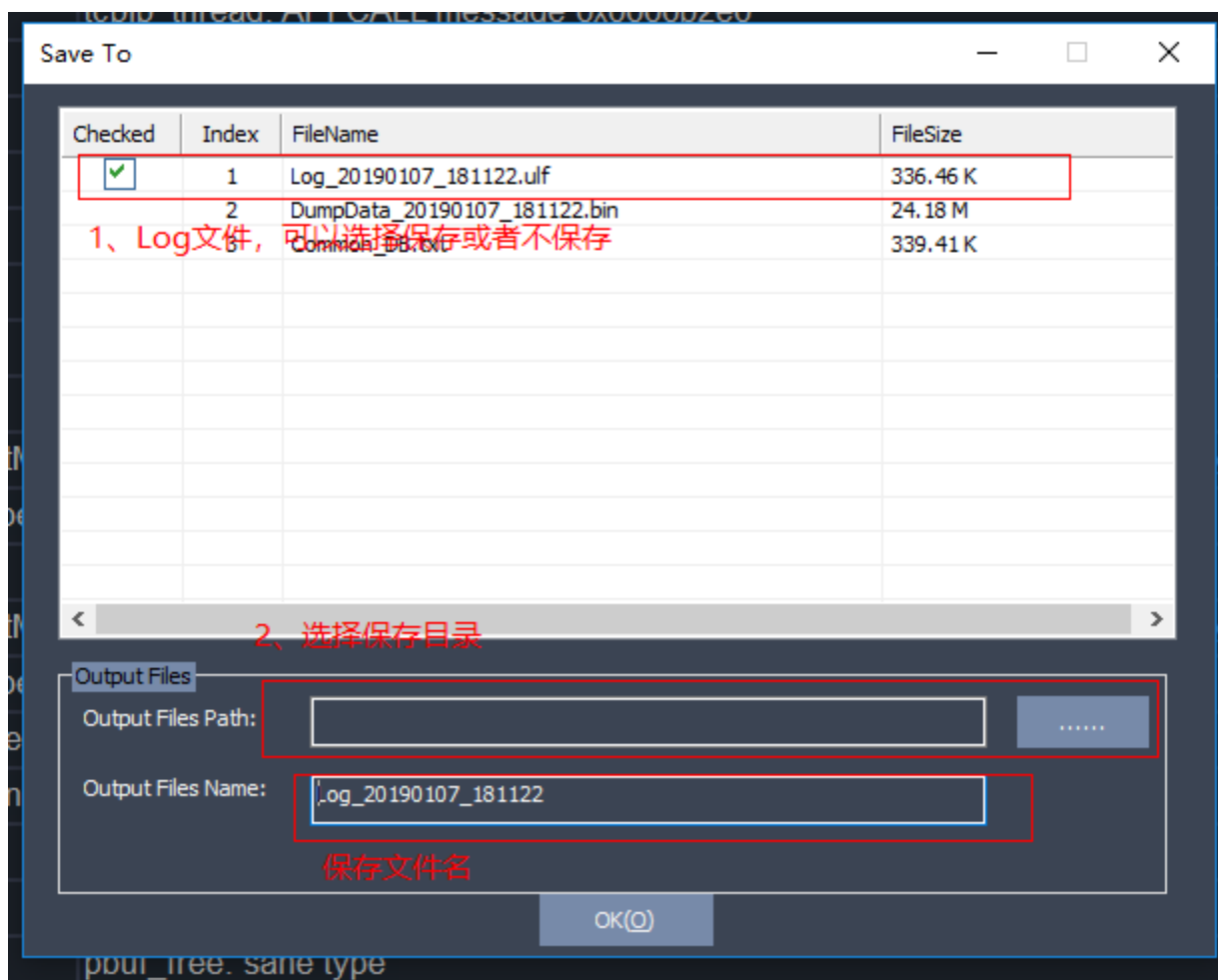


Figure 2-30. Log 保存-2



3. 使用 GCC 工具链开发

*待开发，暂不支持

4. 附录

4.1 参考文档

Table 4-1. 参考文档

序号	文档名称	注释
【1】		
【2】		

4.2 术语和解释

Table 4-2. 术语和解释

术语	解释

4.3 安全和警告

EIGENCOMM CONFIDENTIAL

5. 版本

版本	日期	备注
Draft	2019-03-27	初始版本
V1.1	2020-04-02	根据最新 SDK 更新，并对 EC NB-IoT 芯片通用（EC616/617）

6. 关于我们

上海移芯通信科技有限公司坐落于上海张江，公司于 2017 年 2 月成立，从事蜂窝移动通信芯片的研发和销售。公司产品主要应用于广域物联网通信领域，致力于设计最具性价比的蜂窝物联网芯片。公司团队在蜂窝通信芯片上有着辉煌历史和丰富经验。公司所有核心技术全部自研，包括算法&架构、射频、基带、SoC、协议栈软件、平台&应用软件和硬件方案等。

移芯通信首款自主研发的超低功耗 NB-IoT 芯片 EC616 于 2019 年中量产，被绝大部分头部模组企业采用，现已大批量应用于全国各区域各行业。下一代超高集成度 NB-IoT 芯片 EC616S 于 20 年底量产，超低功耗 4G Cat.1 芯片 EC618 工程样片阶段，5G 芯片正在研发中。

上海移芯通信科技有限公司

地址：中国上海市浦东新区祥科路 298 号佑越国际 1 幢 7 层

邮编：201210

电话：

电邮：

网址：<http://www.eigencomm.com>

技术支持窗口

电邮：support@eigencomm.com